

Linear Models: Classification

Machine Learning (Bilingual)

齐飞 (Fei **Qi**) <fred.qi@ieee.org>

李阳 (Yang **Li**)

March 2024

School of Artificial Intelligence

Xidian University





Linear Models for Classification

Least squares for classification

Perceptron

Logistic Regression

Linear Models for Classification



Goal in classification

To take an input vector $\mathbf{x}$ and to assign it to one of K discrete class $\mathcal{C}_k$ where $k = 1, \dots, K$.

Discriminant Functions

A discriminant is a function that takes an input vector $\mathbf{x}$ and assigns it to one of K classes, denoted $\mathcal{C}_k$.



The representation of a linear discriminant function is obtained by taking a linear function of the input vector so that

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$$

- $\mathbf{w}$ is called a **weight vector**, w_0 is a **bias**



The representation of a linear discriminant function is obtained by taking a linear function of the input vector so that

$$y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$$

- $\mathbf{w}$ is called a **weight vector**, w_0 is a **bias**

Discriminant

- An input vector $\mathbf{x}$ is assigned to class $\mathcal{C}_1$ if $y(\mathbf{x}) \geq 0$
- and to class $\mathcal{C}_2$ otherwise.
- The corresponding decision boundary is therefore defined by the relation $y(\mathbf{x}) = 0$,
- which corresponds to a $(D - 1)$ -dimensional hyperplane within the D -dimensional input space.



- Consider two points $\mathbf{x}_A$ and $\mathbf{x}_B$ both of which lie on the decision surface.
- Because $y(\mathbf{x}_A) = y(\mathbf{x}_B) = 0$, we have $\mathbf{w}^T(\mathbf{x}_A - \mathbf{x}_B) = 0$
- and hence the vector $\mathbf{w}$ is orthogonal to every vector lying within the decision surface,
- and so $\mathbf{w}$ determines **the orientation of the decision surface**.

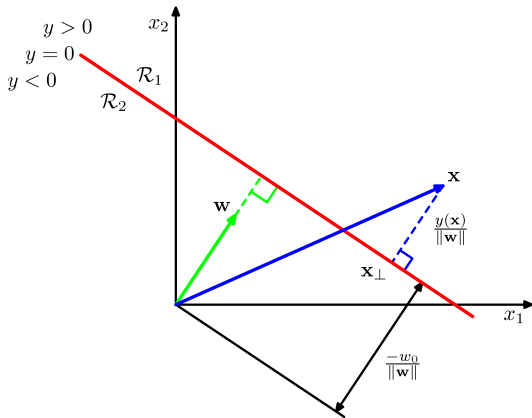
Location of the decision surface



- If $\mathbf{x}$ is a point on the decision surface, then $y(\mathbf{x}) = 0$,
- the normal distance from the origin to the decision surface is given by

$$\frac{\mathbf{w}^T \mathbf{x}}{\|\mathbf{x}\|} = -\frac{w_0}{\|\mathbf{w}\|}$$

- the bias parameter w_0 determines the location of the decision surface.



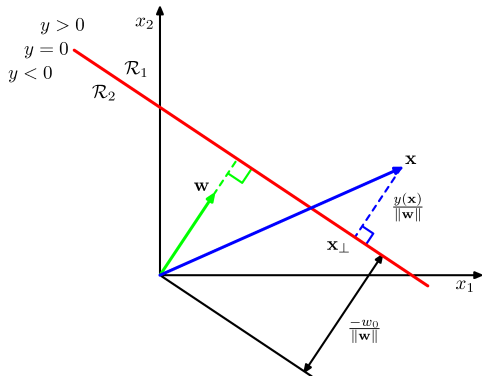
Geometry related to decision surface



- The value of $y(\mathbf{x})$ gives a signed measure of the perpendicular distance r of the point $\mathbf{x}$ from the decision surface.
- Consider an arbitrary point $\mathbf{x}$ and let $\mathbf{x}_\perp$ be its orthogonal projection onto the decision surface

$$\mathbf{x} = \mathbf{x}_\perp + r \frac{\mathbf{w}}{\|\mathbf{w}\|}$$

- Multiplying both sides of this result by $\mathbf{w}^T$ and adding w_0 , making use of $y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$ and $y(\mathbf{x}_\perp) = \mathbf{w}^T \mathbf{x}_\perp + w_0 = 0$, we have



$$r = \frac{y(\mathbf{x})}{\|\mathbf{w}\|}$$

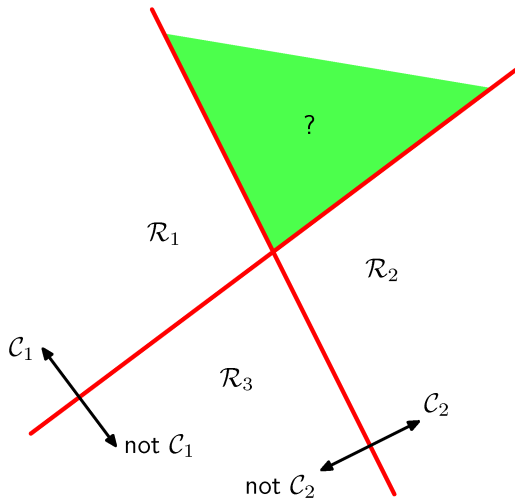


- Now consider the extension of linear discriminants to $K > 2$ classes.
- We might be tempted to build a K-class discriminant by combining a number of two-class discriminant functions.
- However, this leads to some serious difficulties.

One-versus-the-rest classifier



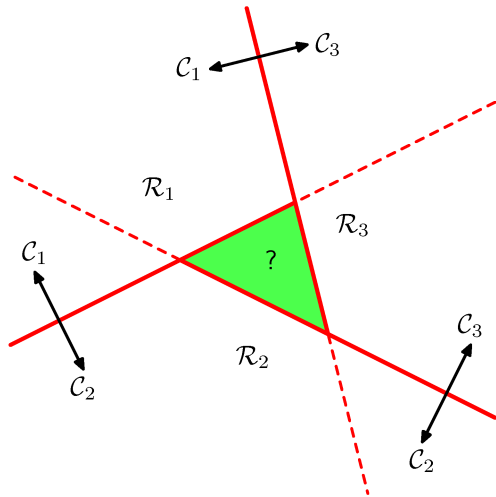
- Consider the use of $K - 1$ classifiers
- each of which solves a two-class problem of separating points in a particular class $\mathcal{C}_k$ from points not in that class
- An example involving three classes where this approach leads to regions of input space that are **ambiguously classified**.



One-versus-one classifier



- An alternative is to introduce $K(K - 1)/2$ binary discriminant functions
- One for every possible pair of classes
- Each point is then classified according to a majority vote amongst the discriminant functions.
- This too runs into the problem of **ambiguous regions**





- A single K-class discriminant comprising K linear functions of the form

$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + w_{k0}$$

- then assigning a point $\mathbf{x}$ to class $\mathcal{C}_k$ if

$$y_k(\mathbf{x}) > y_j(\mathbf{x}) \quad \forall j \neq k$$

- The decision boundary between class $\mathcal{C}_k$ and $\mathcal{C}_j$ is given by $y_k(\mathbf{x}) = y_j(\mathbf{x})$
- corresponds to a $(D - 1)$ -dimensional hyperplane defined by

$$(\mathbf{w}_k - \mathbf{w}_j)^T \mathbf{x} + (w_{k0} - w_{j0}) = 0$$

- This has the same form as the decision boundary for the two-class case.

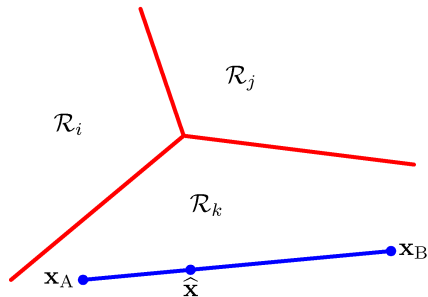
Decision regions are singly connected and convex



- Consider two points $\mathbf{x}_A$ and $\mathbf{x}_B$ both of which lie inside decision region $\mathcal{R}_k$.
- Any point $\hat{\mathbf{x}}$ that lies on the line connecting $\mathbf{x}_A$ and $\mathbf{x}_B$ can be expressed in the form

$$\hat{\mathbf{x}} = \lambda \mathbf{x}_A + (1 - \lambda) \mathbf{x}_B$$

where $0 \leq \lambda \leq 1$.



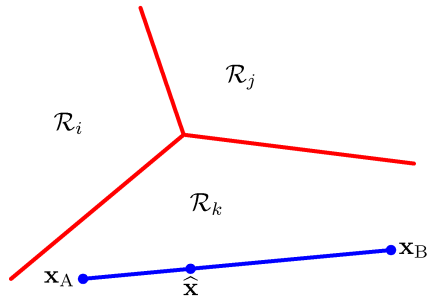
Decision regions are singly connected and convex



- From the linearity of the discriminant functions, it follows that

$$y_k(\hat{\mathbf{x}}) = \lambda y_k(\mathbf{x}_A) + (1 - \lambda)y_k(\mathbf{x}_B)$$

- Because both $\mathbf{x}_A$ and $\mathbf{x}_B$ lie inside $\mathcal{R}_k$, it follows that $y_k(\mathbf{x}_A) > y_j(\mathbf{x}_A)$ and $y_k(\mathbf{x}_B) > y_j(\mathbf{x}_B)$, for all $j \neq k$.
- Hence $y_k(\hat{\mathbf{x}}) > y_j(\hat{\mathbf{x}})$, so $\hat{\mathbf{x}}$ also lies inside $\mathcal{R}_k$.



Least squares for classification



- Each class $\mathcal{C}_k$ is described by its own linear model so that

$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + w_{k0}$$

where $k = 1, \dots, K$.

- We can conveniently group these together using vector notation so that

$$\mathbf{y}(\mathbf{x}) = \tilde{\mathbf{w}}^T \tilde{\mathbf{x}}$$

where $\tilde{\mathbf{w}}$ is a matrix whose k^{th} column comprises $(D + 1)$ -dimensional vector $\tilde{\mathbf{w}}_k = (\mathbf{w}_{k0}, \mathbf{w}_k^T)^T$ and $\tilde{\mathbf{x}}$ is the corresponding **augmented** input vector $(1, \mathbf{x}^T)^T$ with a dummy input $x_0 = 1$.

Determine the parameter matrix



- We now determine the parameter matrix $\tilde{\mathbf{w}}$ by minimizing a sum-of-squares error function.
- Consider a training data set $\{\mathbf{x}_n, \mathbf{t}_n\}$ where $n = 1, \dots, N$, and define a matrix $\mathbf{T}$ whose n^{th} row is the vector $\mathbf{t}_n^T$, together with a matrix $\tilde{\mathbf{x}}$ whose n^{th} row is $\tilde{\mathbf{x}}_n^T$.
- The sum-of-squares error function can then be written as

$$E_D(\tilde{\mathbf{w}}) = \frac{1}{2} \text{Tr}\{(\tilde{\mathbf{x}}\tilde{\mathbf{w}} - \mathbf{T})^T(\tilde{\mathbf{x}}\tilde{\mathbf{w}} - \mathbf{T})\}.$$

Determine the parameter matrix



- Setting the derivative with respect to $\tilde{\mathbf{w}}$ to zero, and rearranging, we then obtain the solution for $\tilde{\mathbf{w}}$ in the form

$$\tilde{\mathbf{w}} = (\tilde{\mathbf{x}}^T \tilde{\mathbf{x}})^{-1} \tilde{\mathbf{x}}^T \mathbf{T} = \tilde{\mathbf{x}}^\dagger \mathbf{T}$$

where $\tilde{\mathbf{x}}^\dagger$ is the pseudo-inverse of the matrix $\tilde{\mathbf{x}}$.

- We then obtain the discriminant function in the form

$$\mathbf{y}(\mathbf{x}) = \tilde{\mathbf{w}}^T \tilde{\mathbf{x}} = \mathbf{T}^T (\tilde{\mathbf{x}}^\dagger)^T \tilde{\mathbf{x}}.$$



A property of least-squares solutions

- An interesting property of least-squares solutions with multiple target variables is that if every target vector in the training set satisfies some linear constraint

$$\mathbf{a}^T \mathbf{t}_n + b = 0$$

for some constants $\mathbf{a}$ and b , then the model prediction for any value of $\mathbf{x}$ will satisfy the same constraint so that

$$\mathbf{a}^T \mathbf{y}(\mathbf{x}) + b = 0.$$

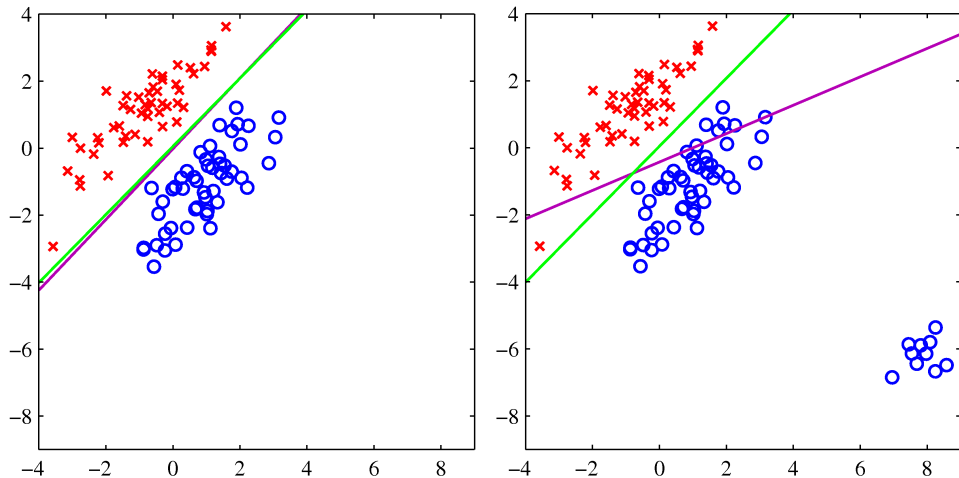
- Thus if we use a 1-of-K coding scheme for K classes, then the predictions made by the model will have the property that the elements of $\mathbf{y}(\mathbf{x})$ will sum to 1 for any value of $\mathbf{x}$.



$$\mathbf{a}^T \mathbf{y}(\mathbf{x}) + b = 0$$

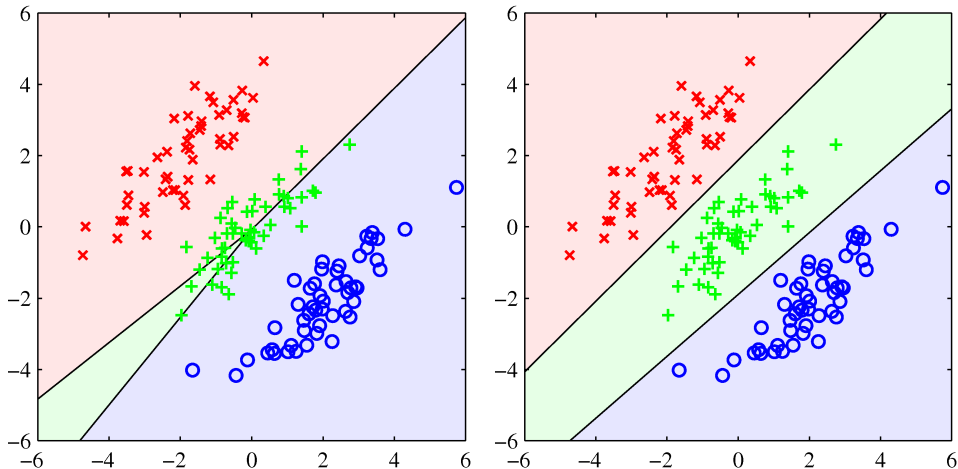
- However, this summation constraint alone is not sufficient to allow the model outputs to be interpreted as probabilities because they are not constrained to lie within the interval $(0, 1)$.

Least-squares solutions lack robustness to outliers



The figures show data from two classes, denoted by red crosses and blue circles.

More severe problems with least-squares



Example of a synthetic data set comprising three classes.

Perceptron

The Mark 1 perceptron hardware (Rosenblatt 1962)



- It occupies an important place in the history of pattern recognition
- Left photo shows how inputs were obtained using a simple camera system.
- A printed character was illuminated by powerful lights.
- An image focused onto a 20×20 array of cadmium sulphide photocells, giving a primitive 400 pixel image.

The Mark 1 perceptron hardware (Rosenblatt 1962)



- The perceptron also had a patch board, shown in the middle photograph, which allowed different configurations of input features to be tried.

The perceptron algorithm



- A two-class model
- The input vector $\mathbf{x}$ is first transformed using a fixed nonlinear transformation to give a feature $\boldsymbol{\varphi}(\mathbf{x})$ vector
- Feature vector is then used to construct a generalized linear model:

$$y(\mathbf{x}) = f(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x})) \quad (\text{Perceptron})$$

- where the nonlinear activation function $f(\cdot)$ is given by a step function:

$$f(a) = \begin{cases} +1, & a \geq 0 \\ -1, & a < 0 \end{cases}$$

- The vector $\boldsymbol{\varphi}(\mathbf{x})$ will typically include a bias component $\varphi_0(\mathbf{x}) = 1$.



- In earlier discussions of two-class classification problems, we have focussed on a target coding scheme in which $t \in \{0, 1\}$, which is appropriate in the context of probabilistic models.
- For the perceptron, however, it is more convenient to use target values $t = +1$ for class $\mathcal{C}_1$ and $t = -1$ for class $\mathcal{C}_2$, which matches the choice of activation function.



- To determine the parameters w
- A natural choice of error function would be the total number of misclassified patterns.
- This leads to a piecewise constant function of w , with discontinuities wherever a change in w causes the decision boundary to move across one of the data points.
- Methods based on changing w using the gradient of the error function cannot then be applied, because the gradient is zero almost everywhere.



- To seek a weight vector $\mathbf{w}$ such that
 - patterns $\mathbf{x}_n$ in class $\mathcal{C}_1$ will have $\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_n) > 0$,
 - whereas patterns $\mathbf{x}_n$ in class $\mathcal{C}_2$ have $\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_n) < 0$.
- Using the $t \in \{-1, +1\}$ target coding scheme it follows that all patterns satisfy

$$\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_n) t_n > 0$$

- The perceptron criterion associates zero error with any pattern that is correctly classified, whereas for a misclassified pattern $\mathbf{x}_n$ it tries to minimize the quantity $-\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_n) t_n$



The **perceptron criterion** is therefore given by

$$E_p(\mathbf{w}) = - \sum_{n \in \mathcal{M}} \mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_n) t_n$$

where $\mathcal{M}$ denotes the set of all misclassified patterns.

- The contribution to the error associated with a particular misclassified pattern is a linear function of $\mathbf{w}$ in regions of $\mathbf{w}$ space where the pattern is misclassified
- and zero in regions where it is correctly classified.
- The total error function is therefore **piecewise linear**.



- We now apply the **stochastic gradient descent** algorithm to this error function.
- The change in the weight vector $\mathbf{w}$ is then given by

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_p(\mathbf{w}) = \mathbf{w}^{(\tau)} + \eta \boldsymbol{\varphi}_n t_n$$

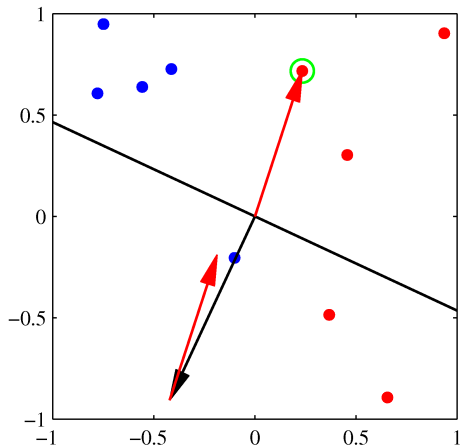
where η is the **learning rate** and τ indexes the steps of the algorithm.

- Because the perceptron function $y(\mathbf{x}, \mathbf{w})$ is unchanged if we multiply $\mathbf{w}$ by a constant, without loss of generality, we set $\eta = 1$.
- Note that, as the weight vector evolves during training, **the set of patterns that are misclassified will change**.



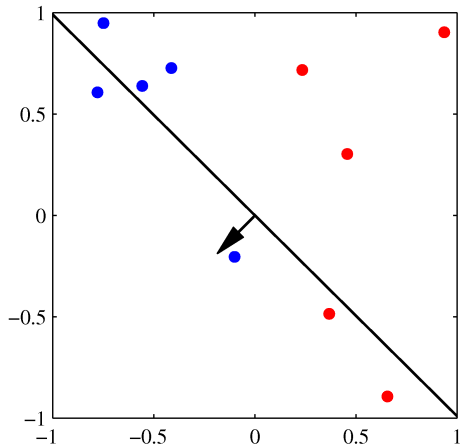
- We cycle through the training patterns in turn, and for each pattern $\mathbf{x}_n$ we evaluate the perceptron function.
- If the pattern is **correctly** classified, then the weight vector remains unchanged
- whereas if it is **incorrectly** classified, then
 - for class $\mathcal{C}_1$ we add the vector $\boldsymbol{\varphi}(\mathbf{x}_n)$ onto the current estimate of weight vector $\mathbf{w}$
 - while for class $\mathcal{C}_2$ we subtract the vector $\boldsymbol{\varphi}(\mathbf{x}_n)$ from $\mathbf{w}$.

Convergence of the perceptron learning algorithm



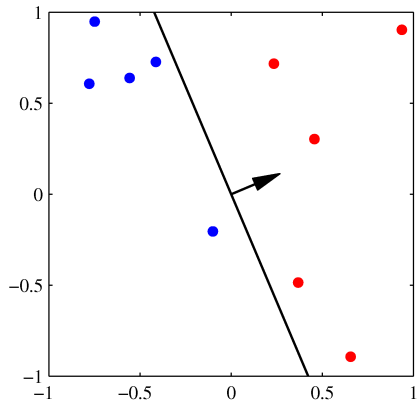
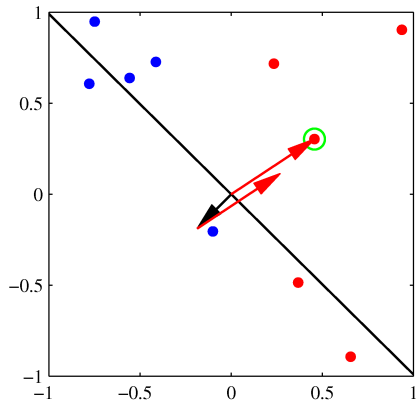
- The plot shows the initial parameter vector w shown as a black arrow together with the corresponding decision boundary (black line), in which the arrow points towards the decision region which is classified as belonging to the red class.
- The data point circled in green is misclassified.

Convergence of the perceptron learning algorithm



- So the feature vector of the misclassified sample is added to the current weight vector, giving the new decision boundary shown in the plot.

Convergence of the perceptron learning algorithm



- Left: next misclassified point to be considered (green circle)
- Right: decision boundary with all data points are correctly classified.



- If we consider the effect of a single update in the perceptron learning algorithm, we see that the contribution to the error from a misclassified pattern will be reduced we have

$$-\mathbf{w}^{(\tau+1)T} \boldsymbol{\varphi}_n \mathbf{t}_n = -\mathbf{w}^{(\tau)T} \boldsymbol{\varphi}_n \mathbf{t}_n - (\boldsymbol{\varphi}_n \mathbf{t}_n)^T \boldsymbol{\varphi}_n \mathbf{t}_n < -\mathbf{w}^{(\tau)T} \boldsymbol{\varphi}_n \mathbf{t}_n$$

- This does not imply that the contribution to the error function from the other misclassified patterns will have been reduced.
- Thus the perceptron learning rule is not guaranteed to reduce the total error function at each stage.



- However, the **perceptron convergence theorem** states that if there exists an exact solution (in other words, if **the training data set is linearly separable**), then the perceptron learning algorithm is guaranteed to find an exact solution in a **finite number** of steps.
- Proofs of this theorem can be found for example in Rosenblatt (1962), Block (1962), Nilsson (1965), Minsky and Papert (1969), Hertz et al. (1991), and Bishop (1995a).



- Note, however, that the number of steps required to achieve convergence could still be substantial, and in practice, until convergence is achieved, we will not be able to distinguish between a nonseparable problem and one that is simply slow to converge.
- Even when the data set is linearly separable, there may be many solutions, and which one is found will depend on the initialization of the parameters and on the order of presentation of the data points.
- Furthermore, for data sets that are not linearly separable, the perceptron learning algorithm will never converge.

Logistic Regression

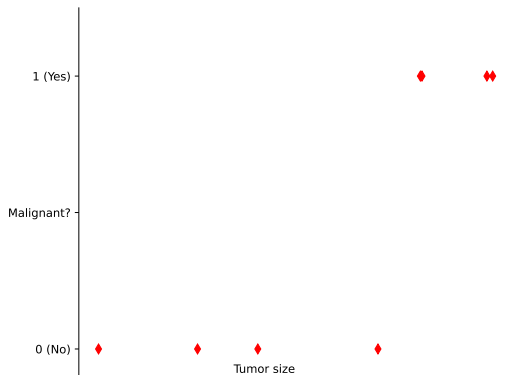


- Email: spam/non-spam
- Online transaction: cheating (Yes/No)
- Tumor: Benign/Malignant

The target variable is discrete

$$t \in \{0, 1\}$$

- “0”: Negative Class (e.g., benign tumor)
- “1”: Positive Class (e.g., malignant tumor)



- To classify the output of $h_w(x)$ with the threshold 0.5:
 - if $h_w(x) > 0.5$, predicting $t = 1$
 - if $h_w(x) < 0.5$, predicting $t = 0$
- The output of classification is either $t = 0$ or 1
 - In regression, the output could be > 1 or < 0

Logistic Regression

$$0 \leq h_w(\mathbf{x}) \leq 1$$

Hypothesis of logistic regression



The logistic regression model expects

$$0 \leq h_{\mathbf{w}}(\mathbf{x}) \leq 1$$

So we represent it with

$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x}).$$

Hypothesis of logistic regression



The logistic regression model expects

$$0 \leq h_{\mathbf{w}}(\mathbf{x}) \leq 1$$

So we represent it with

$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x}).$$

Sigmoid/Logistic function

$$g(z) = \frac{1}{1 + \exp(-z)}.$$

Hypothesis of logistic regression



The logistic regression model expects

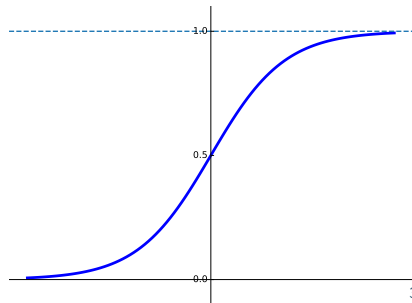
$$0 \leq h_{\mathbf{w}}(\mathbf{x}) \leq 1$$

So we represent it with

$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x}).$$

Sigmoid/Logistic function

$$g(z) = \frac{1}{1 + \exp(-z)}.$$





The hypothesis $h_w(\mathbf{x})$ estimates the probability of the output being $t = 1$ according to the given input $\mathbf{x}$.

Example

If we have

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{Tumorsize} \end{bmatrix}$$
$$h_w(\mathbf{x}) = 0.7$$

It follows that the patient has a probability of 70% to be with malignant tumor.

Probabilistic explanation to logistic model



The hypothesis $h_w(\mathbf{x})$ estimates the probability of the output being $t = 1$ according to the given input $\mathbf{x}$.

Example

If we have

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{Tumorsize} \end{bmatrix}$$
$$h_w(\mathbf{x}) = 0.7$$

It follows that the patient has a probability of 70% to be with malignant tumor.

Probabilistic explanation

$$h_w(\mathbf{x}) = P(t = 1 | \mathbf{x}; \mathbf{w})$$

With parameters $\mathbf{w}$, the probability of the output being $t = 1$ when the input is $\mathbf{x}$.

Probabilistic explanation to logistic model



The hypothesis $h_w(\mathbf{x})$ estimates the probability of the output being $t = 1$ according to the given input $\mathbf{x}$.

Example

If we have

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{Tumorsize} \end{bmatrix}$$
$$h_w(\mathbf{x}) = 0.7$$

It follows that the patient has a probability of 70% to be with malignant tumor.

Probabilistic explanation

$$h_w(\mathbf{x}) = P(t = 1 | \mathbf{x}; \mathbf{w})$$

With parameters $\mathbf{w}$, the probability of the output being $t = 1$ when the input is $\mathbf{x}$.

$$P(t = 0 | \mathbf{x}; \mathbf{w}) + P(t = 1 | \mathbf{x}; \mathbf{w}) = 1$$



The hypothesis $h_w(\mathbf{x})$ estimates the probability of the output being $t = 1$ according to the given input $\mathbf{x}$.

Example

If we have

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{Tumorsize} \end{bmatrix}$$
$$h_w(\mathbf{x}) = 0.7$$

It follows that the patient has a probability of 70% to be with malignant tumor.

Probabilistic explanation

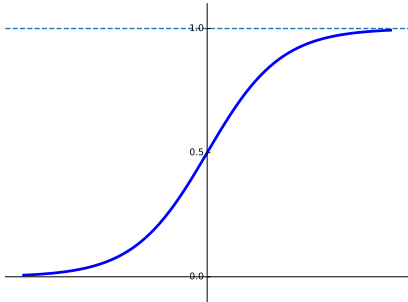
$$h_w(\mathbf{x}) = P(t = 1 | \mathbf{x}; \mathbf{w})$$

With parameters $\mathbf{w}$, the probability of the output being $t = 1$ when the input is $\mathbf{x}$.

$$P(t = 0 | \mathbf{x}; \mathbf{w}) + P(t = 1 | \mathbf{x}; \mathbf{w}) = 1$$

$$P(t = 0 | \mathbf{x}; \mathbf{w}) = 1 - P(t = 1 | \mathbf{x}; \mathbf{w})$$

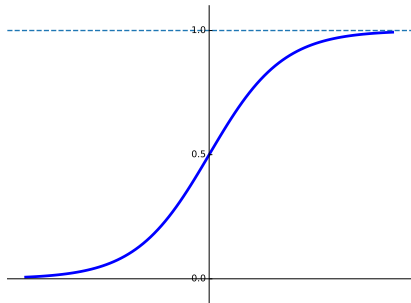
Decision boundary of logistic regression



$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x})$$

$$g(z) = \frac{1}{1 + \exp(-z)}$$

Decision boundary of logistic regression



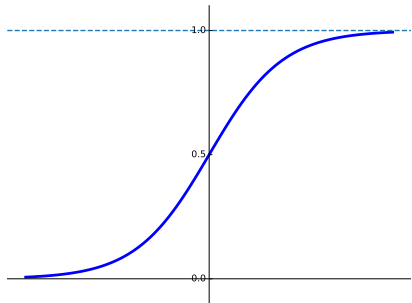
$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x})$$

$$g(z) = \frac{1}{1 + \exp(-z)}$$

So that:

- If $h_{\mathbf{w}}(\mathbf{x}) \geq 0.5$, the model outputs $t = 1$
 - $\mathbf{w}^T \mathbf{x} \geq 0.0$

Decision boundary of logistic regression



$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}^T \mathbf{x})$$

$$g(z) = \frac{1}{1 + \exp(-z)}$$

So that:

- If $h_{\mathbf{w}}(\mathbf{x}) \geq 0.5$, the model outputs $t = 1$
 - $\mathbf{w}^T \mathbf{x} \geq 0.0$
- If $h_{\mathbf{w}}(\mathbf{x}) < 0.5$, the model outputs $t = 0$
 - $\mathbf{w}^T \mathbf{x} < 0.0$

Linear decision boundary



$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}_0 + \mathbf{w}_1 x_1 + \mathbf{w}_2 x_2),$$

$$\mathbf{w} = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}.$$



$$h_{\mathbf{w}}(\mathbf{x}) = g(w_0 + w_1x_1 + w_2x_2),$$

$$\mathbf{w} = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}.$$

Then the condition for the output being $t = 1$ is that

$$-3 + x_1 + x_2 \geq 0,$$

i.e.,

$$x_1 + x_2 \geq 3$$

Linear decision boundary



$$h_{\mathbf{w}}(\mathbf{x}) = g(\mathbf{w}_0 + \mathbf{w}_1 x_1 + \mathbf{w}_2 x_2),$$

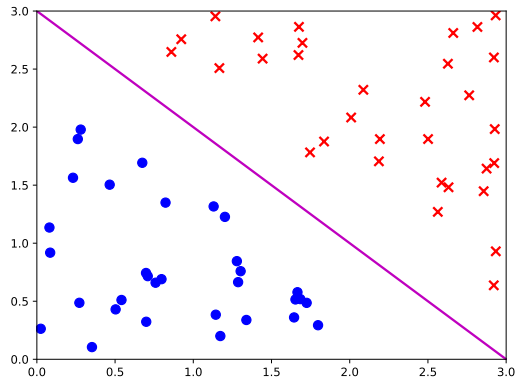
$$\mathbf{w} = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}.$$

Then the condition for the output being $t = 1$ is that

$$-3 + x_1 + x_2 \geq 0,$$

i.e.,

$$x_1 + x_2 \geq 3$$



Non-linear (circular) decision boundary



$$h_{\mathbf{w}}(\mathbf{x}) = g(w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2)$$

$$\mathbf{w} = \begin{bmatrix} -1 & 0 & 0 & 1 & 1 \end{bmatrix}^T$$



Non-linear (circular) decision boundary

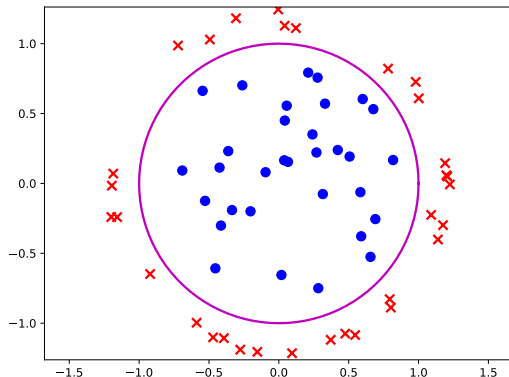
$$h_{\mathbf{w}}(\mathbf{x}) = g(w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2)$$

$$\mathbf{w} = \begin{bmatrix} -1 & 0 & 0 & 1 & 1 \end{bmatrix}^T$$

Then the condition for the output being $t = 1$ is that

$$-1 + x_1^2 + x_2^2 = 0,$$

$$\text{i.e.,} \quad x_1^2 + x_2^2 = 1$$





Non-linear (circular) decision boundary

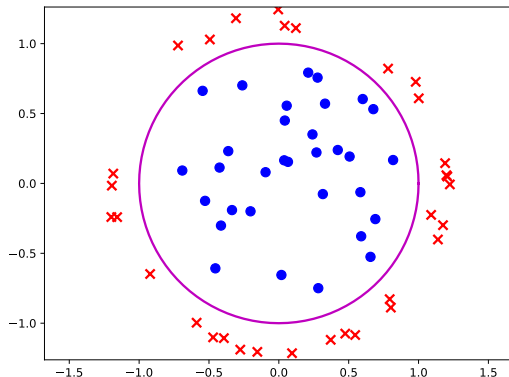
$$h_{\mathbf{w}}(\mathbf{x}) = g(w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2)$$

$$\mathbf{w} = \begin{bmatrix} -1 & 0 & 0 & 1 & 1 \end{bmatrix}^T$$

Then the condition for the output being $t = 1$ is that

$$-1 + x_1^2 + x_2^2 = 0,$$

$$\text{i.e.,} \quad x_1^2 + x_2^2 = 1$$



$$h_{\mathbf{w}} = g(w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1^2x_2 + w_5x_1^2x_2^2 + w_6x_1^3x_2 + \dots)$$



The training set is with m samples,

$$\{(\mathbf{x}^{(1)}, t^{(1)}), (\mathbf{x}^{(2)}, t^{(2)}), \dots, (\mathbf{x}^{(m)}, t^{(m)})\}$$

where

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n+1} \quad x_0 = 1, \quad t \in \{0, 1\}$$

The hypothesis is

$$h_{\mathbf{w}} = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})}$$

How to get parameters $\mathbf{w}$?



$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)})$$

- Linear regression

$$\begin{aligned} \text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)}) &= \frac{1}{2} (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - t^{(i)})^2 \\ &= \frac{1}{2} (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)})^2 \end{aligned}$$

- Logistic regression

$$\text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)}) = \frac{1}{2} \left(\frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}^{(i)})} - t^{(i)} \right)^2$$

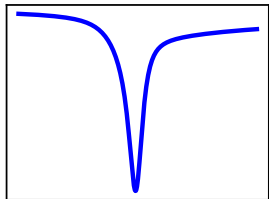
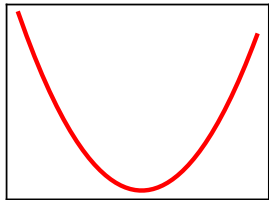
$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)})$$

- Linear regression

$$\begin{aligned} \text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)}) &= \frac{1}{2} (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - t^{(i)})^2 \\ &= \frac{1}{2} (\mathbf{w}^T \mathbf{x}^{(i)} - t^{(i)})^2 \end{aligned}$$

- Logistic regression

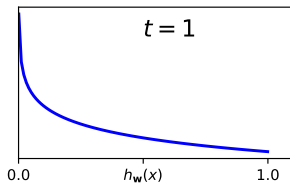
$$\text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)}) = \frac{1}{2} \left(\frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x}^{(i)})} - t^{(i)} \right)^2$$





$$\text{cost}(h_{\mathbf{w}}(\mathbf{x}), t) = \begin{cases} -\log h_{\mathbf{w}}(\mathbf{x}) & \text{if } t = 1 \\ -\log(1 - h_{\mathbf{w}}(\mathbf{x})) & \text{if } t = 0 \end{cases}$$

- When $t = 1$ and $h_{\mathbf{w}}(\mathbf{x}) = 1.0$, the cost function is 0.
- When $h_{\mathbf{w}}(\mathbf{x}) \rightarrow 0$, the cost function approaches to ∞ .
- When $h_{\mathbf{w}}(\mathbf{x}) = 0$ but $t = 1$, the learning algorithm will be heavily punished by this cost function.

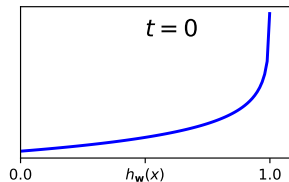
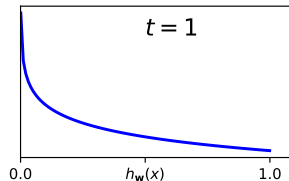


Cost function



$$\text{cost}(h_{\mathbf{w}}(\mathbf{x}), t) = \begin{cases} -\log h_{\mathbf{w}}(\mathbf{x}) & \text{if } t = 1 \\ -\log(1 - h_{\mathbf{w}}(\mathbf{x})) & \text{if } t = 0 \end{cases}$$

- When $t = 1$ and $h_{\mathbf{w}}(\mathbf{x}) = 1.0$, the cost function is 0.
- When $h_{\mathbf{w}}(\mathbf{x}) \rightarrow 0$, the cost function approaches to ∞ .
- When $h_{\mathbf{w}}(\mathbf{x}) = 0$ but $t = 1$, the learning algorithm will be heavily punished by this cost function.



Simplification of the cost function



$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)})$$

$$\text{cost} (h_{\mathbf{w}}(\mathbf{x}), t) = \begin{cases} -\log h_{\mathbf{w}}(\mathbf{x}) & \text{if } t = 1 \\ -\log(1 - h_{\mathbf{w}}(\mathbf{x})) & \text{if } t = 0 \end{cases}$$

Simplification of the cost function



$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)})$$

$$\text{cost} (h_{\mathbf{w}}(\mathbf{x}), t) = \begin{cases} -\log h_{\mathbf{w}}(\mathbf{x}) & \text{if } t = 1 \\ -\log(1 - h_{\mathbf{w}}(\mathbf{x})) & \text{if } t = 0 \end{cases}$$

Considering that $t = 1$ or $t = 0$, we have

$$\text{cost} (h_{\mathbf{w}}(\mathbf{x}), t) =$$

Simplification of the cost function



$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)})$$

$$\text{cost} (h_{\mathbf{w}}(\mathbf{x}), t) = \begin{cases} -\log h_{\mathbf{w}}(\mathbf{x}) & \text{if } t = 1 \\ -\log(1 - h_{\mathbf{w}}(\mathbf{x})) & \text{if } t = 0 \end{cases}$$

Considering that $t = 1$ or $t = 0$, we have

$$\text{cost} (h_{\mathbf{w}}(\mathbf{x}), t) = -t \log h_{\mathbf{w}}(\mathbf{x})$$

Simplification of the cost function



$$J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)})$$

$$\text{cost} (h_{\mathbf{w}}(\mathbf{x}), t) = \begin{cases} -\log h_{\mathbf{w}}(\mathbf{x}) & \text{if } t = 1 \\ -\log(1 - h_{\mathbf{w}}(\mathbf{x})) & \text{if } t = 0 \end{cases}$$

Considering that $t = 1$ or $t = 0$, we have

$$\text{cost} (h_{\mathbf{w}}(\mathbf{x}), t) = -t \log h_{\mathbf{w}}(\mathbf{x}) - (1 - t) \log(1 - h_{\mathbf{w}}(\mathbf{x}))$$



$$\begin{aligned} J(\mathbf{w}) &= \frac{1}{m} \sum_{i=1}^m \text{cost} (h_{\mathbf{w}}(\mathbf{x}^{(i)}), t^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^m [t^{(i)} \log h_{\mathbf{w}}(\mathbf{x}^{(i)}) + (1 - t^{(i)}) \log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)}))] \end{aligned}$$

- To find parameters $\mathbf{w}$ such that

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} J(\mathbf{w})$$

- To predict for a unseen input $\mathbf{x}$

$$h_{\mathbf{w}}(\mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x})} \propto P(t = 1 | \mathbf{x}; \mathbf{w})$$



$$J(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^m \left[t^{(i)} \log h_{\mathbf{w}}(\mathbf{x}^{(i)}) + (1 - t^{(i)}) \log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \right]$$

Minimize $J(\mathbf{w})$

Repeat until converge {

$$w_j := w_j - \alpha \frac{\partial}{\partial w_j} J(\mathbf{w})$$

}

Remember to **simultaneously update** all w_j .



$$J(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^m \left[t^{(i)} \log h_{\mathbf{w}}(\mathbf{x}^{(i)}) + (1 - t^{(i)}) \log(1 - h_{\mathbf{w}}(\mathbf{x}^{(i)})) \right]$$

$$\frac{\partial}{\partial w_j} J(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - t^{(i)}) x_j^{(i)}$$

Repeat until convergence {

$$w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\mathbf{w}}(\mathbf{x}^{(i)}) - t^{(i)}) x_j^{(i)}$$

}

This algo. looks the same to linear regression.